

Шпаргалка по PowerShell

Командлеты и конвейер объектов, файлы и реестр, процессы и службы, сеть, скрипты и функции, удалённое выполнение. Примеры для PowerShell 5.1 и PowerShell 7.

Как устроены команды

Имя командлета — это пара «Глагол-Существительное»: Get-Process, Set-Location, New-Item. По конвейеру передаются объекты, а не текст, поэтому у результата есть свойства.

`Get-Command *service*`

Найти командлет по части имени

`Get-Help Get-Process -Examples`

Справка с примерами (-Full — целиком, -Online — в браузере)

`Get-Member`

Свойства и методы объекта: `Get-Process` | `Get-Member`

`Get-Alias ls`

Что скрывается за коротким именем

`$PSVersionTable`

Версия PowerShell и платформы

Конвейер

`Where-Object { $_.CPU -gt 100 }`

Фильтр; \$_ — текущий объект. Краткая форма: `Where CPU -gt 100`

`Select-Object Name, Id -First 10`

Выбрать поля и первые строки

`Sort-Object CPU -Descending`

Сортировка

`ForEach-Object { $_.Name.ToUpper() }`

Действие над каждым объектом

`Group-Object Status`

Сгруппировать и посчитать

`Measure-Object Length -Sum -Average`

Суммы и средние по числовому свойству

`Format-Table -AutoSize / Format-List`

Вывод таблицей или списком — только в самом конце конвейера

Файлы и каталоги

`Get-ChildItem C:\logs -Recurse -Filter *.log`

Обход дерева каталогов с фильтром (алиасы: `dir`, `ls`)

`Get-Content app.log -Tail 50 -Wait`

Последние строки и слежение за файлом

`Set-Content / Add-Content`

Записать файл заново; дописать в конец

`Copy-Item, Move-Item, Remove-Item`

Копирование, перемещение, удаление (-Recurse - Force)

`New-Item -ItemType Directory -Path C:\app\data`

Создать каталог

`Test-Path C:\app\config.json`

Проверить существование

`Select-String -Pattern "error" -Path *.log`

Поиск по содержимому — аналог `grep`

`Get-FileHash file.iso -Algorithm SHA256`

Контрольная сумма

Процессы и службы

`Get-Process -Name chrome`

Процессы по имени

`Stop-Process -Id 1234 -Force`

Завершить процесс

`Start-Process notepad -ArgumentList file.txt`

Запустить программу

`Get-Service | Where Status -eq Running`

Запущенные службы

`Restart-Service spooler`

Перезапуск службы (есть `Start`, `Stop`, `Set`)

`Get-EventLog -LogName System -Newest 20`

Журнал событий; в PowerShell 7 — `Get-WinEvent`

Сеть

`Test-NetConnection example.com -Port 443`

Проверка доступности узла и порта

`Invoke-WebRequest https://example.com -OutFile page.html`

Скачать страницу или файл

`Invoke-RestMethod https://api.example.com/users`

Запрос к API: JSON сразу превращается в объекты

`Get-NetIPAddress / Get-NetTCPConnection`

Адреса интерфейсов и открытые соединения

`Resolve-DnsName example.com`

Запрос к DNS

Переменные и типы

`$name = "Аня"; $count = 10`

Переменная; тип определяется автоматически

`[int]$x = "42"`

Явное приведение типа

`$list = @(1, 2, 3)`

Массив; `$list += 4` создаёт новый

`$map = @{ name = "Аня"; age = 30 }`

Хеш-таблица; обращение `$map.name`

`"Привет, $name! Всего: $($list.Count)"`

Подстановка в двойных кавычках; в одинарных — текст как есть

`$env:PATH`

Переменные окружения

Условия и циклы

```
if ($count -gt 10) { "много" }
elseif ($count -eq 0) { "пусто" }
else { "мало" }

foreach ($item in $list) { $item }

for ($i = 0; $i -lt 5; $i++) { $i }

while ($queue.Count) { $queue.Pop() }

switch ($status) {
    "ok"      { "готово" }
    "fail"    { "ошибка" }
    default   { "неизвестно" }
}
```

Сравнения записываются словами: -eq -ne -gt -lt -ge -le -like -match -contains -in. Отрицание — -not или !, объединение — -and, -or.

Скрипты и функции

```
function Get-BigFiles {
    param(
        [Parameter(Mandatory)]
        [string]$Path,
        [int]$MinMB = 100
    )

    Get-ChildItem $Path -Recurse -File |
        Where-Object { $_.Length -gt $MinMB * 1MB } |
        Sort-Object Length -Descending
}

Get-BigFiles -Path C:\data -MinMB 500
```

.\script.ps1 -Param значение

Запуск скрипта из текущего каталога

Get-ExecutionPolicy / Set-ExecutionPolicy RemoteSigned
-Scope CurrentUser

Политика запуска скриптов: по умолчанию выполнение часто запрещено

```
try { ... } catch { $_.Exception.Message } finally {
... }
```

Обработка ошибок

-ErrorAction Stop

Сделать ошибку командлета перехватываемой в catch

-WhatIf / -Confirm

Показать, что произойдёт, не выполняя; спросить подтверждение

Данные и форматы

Import-Csv users.csv | Where Age -gt 30

CSV превращается в объекты со свойствами по заголовкам

Export-Csv out.csv -NoTypeInfo -Encoding UTF8
Выгрузка результата в CSV

Get-Content data.json | ConvertFrom-Json
Разбор JSON в объекты

\$obj | ConvertTo-Json -Depth 5
Обратное преобразование; глубина по умолчанию мала

Out-File -Encoding UTF8 / Out-GridView
Запись в файл; интерактивная таблица в окне

Удалённое выполнение

Enter-PSSession -ComputerName srv1

Интерактивный сеанс на другом компьютере

Invoke-Command -ComputerName srv1, srv2 -ScriptBlock {
Get-Service }

Выполнить блок команд сразу на нескольких узлах

\$cred = Get-Credential

Запросить логин и пароль, передаётся параметром -Credential

New-PSSession / Remove-PSSession

Постоянный сеанс для серии команд

Полезные привычки

- Сначала посмотрите на объект через Get-Member — станет ясно, какие свойства можно фильтровать.
- Format-* ставится только в конец: после него в конвейере остаётся текст, а не объекты.
- Опасные действия проверяйте с -WhatIf.
- Модули ставятся из галереи: Install-Module имя -Scope CurrentUser.
- PowerShell 7 кроссплатформенный, команда запуска — pwsh; часть Windows-командлетов в нём недоступна.